

1. Planejamento da capacitação de Git e GitHub

- Realização em 2 dias, 1:00 a 1:30 de duração;
- Abordar conceitos sobre Git e GitHub.

1.1. Conteúdo programado

Dia	Capítulo	Conteúdo
1º	Introdução	Introdução
1º	Git	Instalação
1º	Git	Comandos básicos + Fluxo de trabalho
1º	GitHub	Overview + Comandos de repositório remoto
2º	GitHub	Intermediário
2º	Git + GitHub	Intermediário

1.1.1. Introdução

Explicação sobre o Git, plataformas de hospedagem de repositórios e suas diferenças.

1.1.2. Instalação

Instalação do Git em ambiente Windows, Linux e MacOS.

1.1.3. Comandos básicos + Fluxo de trabalho

Apresentação dos comandos `init`, `config`, `add`, `commit`, `status`, `log` e `checkout`; Explicação sobre o fluxo de trabalho do Git.

1.1.4. Overview da plataforma GitHub + Comandos de repositório remoto

Apresentação da plataforma; Criação de repositórios; Comandos `clone`, `push` e `pull`.

1.1.5. GitHub intermediário

Abordagem sobre o uso da ferramenta *Projects*; *Issues*; *Review* e *Pull Requests*; Configuração do projeto;

1.1.6. Git + GitHub intermediário

Conceito de *branches*; Conceito de *merge*; Conceito de *rebase*; Ocorrência de conflitos e correções; Desfazer operações; .

Introdução

Apresentação

Sou Vitor de Oliveira Veras, aluno de Ciência da Computação, atualmente no 8º período. Nessa capacitação veremos um pouco sobre a ferramenta de versionamento de código Git em conjunto com a plataforma GitHub para hospedagem de código remota.

Ela foi estruturada para ter duração de 2 dias, cada encontro com duração de 01:00 a 01:30.

No primeiro dia vamos apresentar conceitos básicos relacionados a Git e GitHub, no segundo serão aplicadas conceitos mais intermediários.

Sobre o Git

O Git surge como um sistema de controles de versões (VCS) a partir de uma necessidade do projeto do núcleo (kernel) do Linux usar um Sistema de Controle de Versão Distribuído (DVCS). O BitKeeper que passou a ser pago e assim as relações entre o projeto do kernel e a empresa foi desfeita.

Um exemplo de uso é quando temos a necessidade de controlar o processo de mudanças de um arquivo, fiz a mudança em um dia, depois disso fiz outra, mas para isso seria necessário manter duas versões do arquivo na máquina, o que a longo prazo pode gerar confusões. O git então permite fazer uma versão para cada parte desenvolvida, possibilitando assim manter um histórico de alterações.

Sobre plataformas de hospedagem de repositórios

São plataformas que permitem que pessoas contribuam com projetos de código aberto/privados. Alguns exemplos atuais são GitHub, GitLab, BitBucket e Gitea (OpenSource), ou seja, existe um ponto onde todos os participantes podem conferir novas mudanças e controlar a versão para os outros.

Instalação

Em um terminal use o comando `git help` para checar se o binário do git se encontra instalado no sistema.

Para realizar a instalação do Git vai depender do Sistema operacional que está sendo utilizado. Para quem está utilizando Windows basta seguir por esse link, caso você esteja utilizando Linux será por esse link e para MacOS será esse link e seguir as instruções da página.

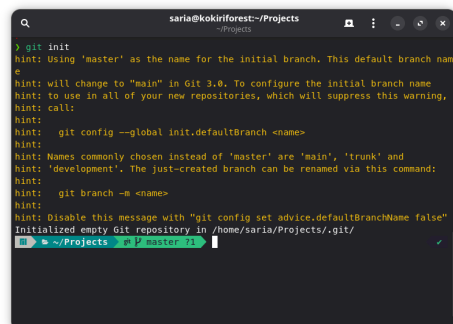
Como sugestão de editor de código queremos o VSCode, mas qualquer outro pode ser utilizado.

Comandos básicos e fluxo de trabalho

Vamos inciar montando a estrutura inicial do nosso projeto a ser utilizado:

git init

No lugar de preferência crie uma pasta com nome `GitGitHub`; Abra o terminal na pasta em questão e então digite o seguinte comando para iniciar o versionamento dos arquivos presentes na pasta: `git init`;



```
saria@kokiriforest:~/Projects
~/Projects
$ git init
hint: Using 'master' as the name for the initial branch. This default branch name
hint: will change to 'main' in Git 3.0. To configure the initial branch name
hint: to use in all of your new repositories, which will suppress this warning,
hint: call:
hint:   git config --global init.defaultBranch <name>
hint: Names commonly chosen instead of 'master' are 'main', 'trunk' and
hint: 'development'. The just-created branch can be renamed via this command:
hint:   git branch -m <name>
Initialized empty Git repository in /home/saria/Projects/.git/
```

Conceito do **Fluxo de trabalho**: O Git trabalha com um conceito de **Estados**, são eles:

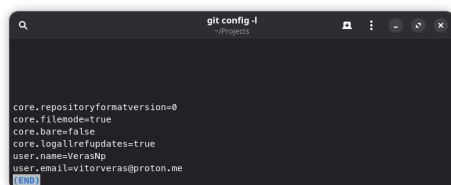
1. Untracked (Não rastreável): Arquivos que não estão sendo monitorados pelo Git;
2. Tracked (Rastreável): Arquivos que estão sob o monitoramento do Git;
 1. Unmodified (Não modificado): Arquivo se encontra atualmente sem nenhuma mudança observada pelo Git;
 2. Modified (Modificado): Arquivo tem novas alterações identificadas pelo Git;
 3. Staged (Preparado): Arquivo se encontra em uma área do Git pronto para consolidar essas alterações.

git config

Use o comando `git config -l` para ver as configurações válidas do git no projeto.

Use os comandos: `git config user.name "<nome>"` e `git config user.email "<email>"` para informar o pertencimento de cada mudança que ele identificar.

Essas configurações servem principalmente para informações da pessoa que fará o commit, como email e usuário, entre outras informações utilizadas pela ferramenta Git.



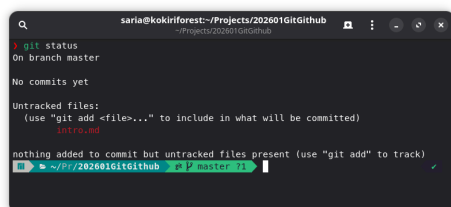
```
git config -l
core.repositoryformatversion=0
core.filemode=true
core.bare=false
core.logallrefupdates=true
user.name=VerasNp
user.email=vtorveras@proton.me
```

git status

Crie um arquivo *Markdown* chamado `intro.md` para iniciar o trabalho

Use o comando `git status` para observar o estado do arquivo até o momento em relação ao Git.

Até o momento o Git não está rastreando o arquivo, logo o estado que se encontra é *Untracked* (Não rastreado).



```
saria@kokiriforest:~/Projects/202601GitGithub
$ git status
On branch master

No commits yet

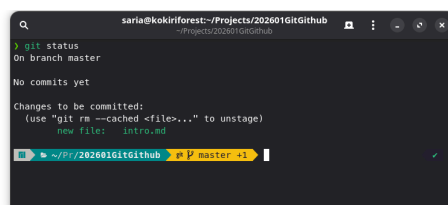
Untracked files:
  (use "git add <file>..." to include in what will be committed)
        intro.md

nothing added to commit but untracked files present (use "git add" to track)
```

Nem todo o arquivo do projeto precisa necessariamente ser rastreado pelo Git, o que veremos mais adiante uma técnica para justamente ignorar alguns arquivos ou pastas desejadas.

git add

Adicione o arquivo ao versionamento com Git usando o comando `git add intro.md`



```
saria@kokiriforest:~/Projects/202601GitGithub
$ git status
On branch master

No commits yet

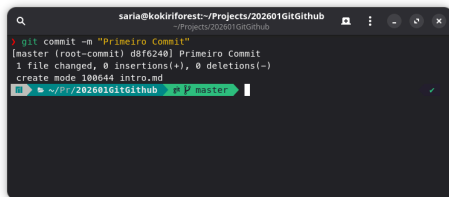
Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   intro.md
```

Ao usar o comando `git status` perceba que a mensagem muda para informar que há mudanças a serem consolidadas, ou seja, o arquivo passou para um estado *Tracked e staged*.

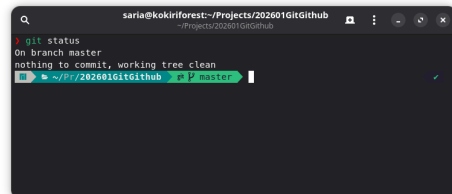
git commit

Vamos consolidar fazendo então o “commit” com o comando `git commit -m "Primeiro commit"`. O Git agora pode consolidar as mudanças ao usar o comando `git commit`.

Ao fazer o commit o git cria um código único para cada commit.



```
saria@kokiriforest:~/Projects/202601GitGithub
~/Projects/202601GitGithub
$ git commit -m "Primeiro Commit"
[master (root-commit) d8f6240] Primeiro Commit
1 file changed, 0 insertions(+), 0 deletions(-)
create mode 100644 intro.md
$
```



```
saria@kokiriforest:~/Projects/202601GitGithub
~/Projects/202601GitGithub
$ git status
On branch master
nothing to commit, working tree clean
$
```

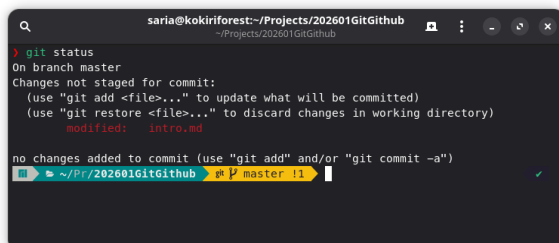
Para observar outro possível estado de um arquivo em estado *Tracked* adicione em `intro.md` o seguinte conteúdo

```
1 # Introdução
2
3 Batata quente
```

md

O arquivo agora se encontra em um estado *Tracked e modified*

Observe o que acontece quando executamos `git status`



```
saria@kokiriforest:~/Projects/202601GitGithub
~/Projects/202601GitGithub
$ git status
On branch master
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   intro.md

no changes added to commit (use "git add" and/or "git commit -a")
$
```

Para então consolidar a mudança do conteúdo execute `git commit -m "Mudança"`.

git log

O git log pode ser utilizado para ver o histórico de commits realizados, trazendo também informações como autoria e código único do commit.

```
git log
~/Projects/202601/GitGithub
Hash Único por commit
commit 4235ea9b2ad6bd90342345aa16924764bc7cf1cd (HEAD -> master)
Author: VerasNp <vitorveras@proton.me>
Date: Wed Jun 3 16:37:31 2026 -0300

Mudanças

commit d8f62403cdcc48c2bc861c9f668637319a0627f4
Author: VerasNp <vitorveras@proton.me>
Date: Wed Jun 3 16:22:13 2026 -0300

Primeiro Commit

~
~
~
(END)
```

git checkout

Pode ser usado para navegar entre versões consolidadas.

Para voltar ao “normal” basta executar `git checkout <branch>`.

Overview GitHub

O Github é uma plataforma lançada em 2008 para um controle de versão distribuído.

Dentro dele é possível gerenciar os repositórios, conjunto de código versionado, onde é possível o gerenciamento de visibilidade do projeto, visualização de commits, versões, solicitações de pull, gerência de projetos e outras atividades que ajudam na gerência de um projeto colaborativo.

Repositórios

Os repositórios são o local onde reside do código fonte, ele pode ser privado ou público. Dentre as terminologias que são usadas:

Nome	Significado
Branch	Uma versão paralela do código contido no repositório
Clone	Download de todo o código presente no repositório
Fork	Criação de um novo repositório que compartilha o mesmo código, configurações de visibilidade do repositório “upstream”
Merge	Pegar mudanças de uma branch e aplicar na outra
Pull Request	Um pedido de merge de uma branch para outra
Remote	Repositório armazenado no GitHub e não em seu computador
Upstream	Branch no repositório original que foi realizado o fork ou clone, a branch correspondente no repositório clonado ou forkado se chama “downstream”

Criação de repositório

Em sua conta crie um novo repositório, escolha um nome, deixe ele público se desejar.

Algumas configurações iniciais são possíveis, dentre elas:

- Escolha de visibilidade;

README.md

Uma descrição do projeto

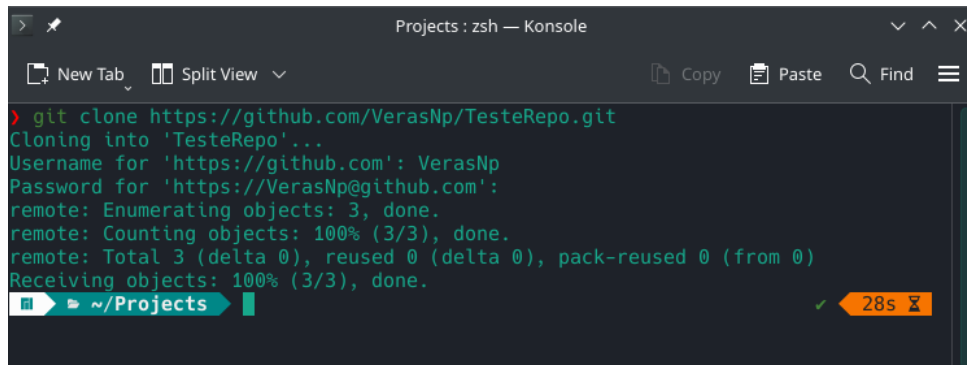
É possível fazer a edição diretamente pelo GitHub.

O .gitignore

A partir dele é possível especificar arquivos os quais não devem ser versionados pelo Git.

Sincronização com git clone

A partir do repositório criado vamos fazer a sincronização usando o comando `git clone`. Em uma pasta, abra o terminal e execute `git clone <url>`.



```
Projects : zsh — Konsole
New Tab Split View Copy Paste Find
> git clone https://github.com/VerasNp/TesteRepo.git
Cloning into 'TesteRepo'...
Username for 'https://github.com': VerasNp
Password for 'https://VerasNp@github.com':
remote: Enumerating objects: 3, done.
remote: Counting objects: 100% (3/3), done.
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (3/3), done.
```

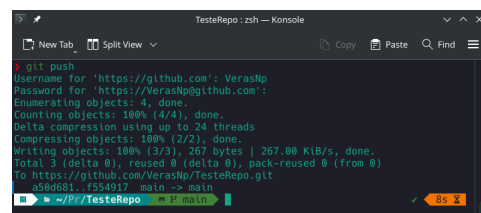
Envio de alterações com git push

Vamos adicionar algo para que possamos assim enviar alterações para o nosso repositório remoto no GitHub.

Na pasta clonada crie um arquivo `index.md`, faça o `git status`, `git add index.md`, `git commit -m "Adição de arquivo"`.

Agora use o comando `git push` para enviar as alterações para o repositório criado no GitHub.

Cheque na página que os arquivos existente localmente foram enviados e com isso você poderá fazer o download dele em outra máquina.



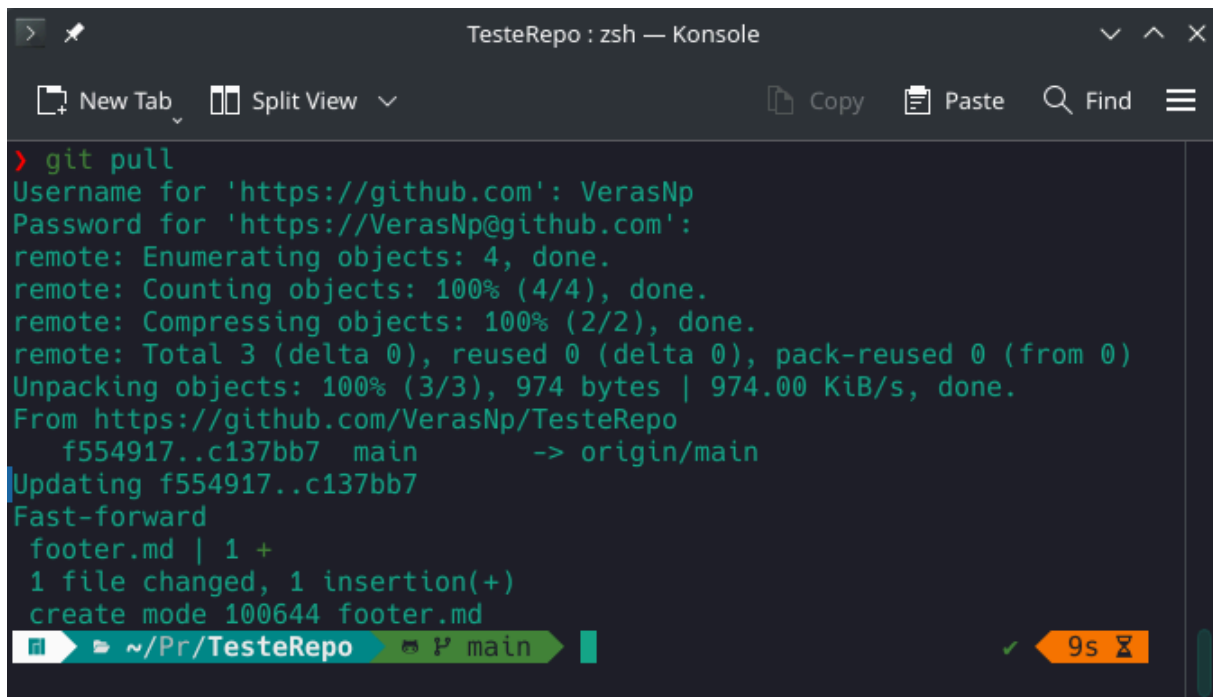
```
TesteRepo : zsh — Konsole
New Tab Split View Copy Paste Find
> git push
Username for 'https://github.com': VerasNp
Password for 'https://VerasNp@github.com':
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 24 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 267 bytes | 267.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/VerasNp/TesteRepo.git
   590581..f5549d7  main -> main
```

Atualizando o local com git pull

A partir do GitHub faremos a criação de um arquivo e então atualizar o repositório local.

No GitHub crie um `footer.md` e commite.

No repositório local perceba a não existência do arquivo que foi criado, assim vamos usar o comando `git pull` para trazer informações de novas mudanças.



A terminal window titled 'TesteRepo : zsh — Konsole' showing the execution of a `git pull` command. The terminal output indicates a successful pull from the remote repository 'https://github.com/VerasNp/TesteRepo'. It shows the process of enumerating, counting, and compressing objects, followed by unpacking. The pull is a fast-forward update from commit `f554917..c137bb7` on the `main` branch. A new file, `footer.md`, is created. The terminal interface includes a top bar with window controls and a bottom status bar showing the current directory as `~/Pr/TesteRepo` and the active branch as `main`.

```
> git pull
Username for 'https://github.com': VerasNp
Password for 'https://VerasNp@github.com':
remote: Enumerating objects: 4, done.
remote: Counting objects: 100% (4/4), done.
remote: Compressing objects: 100% (2/2), done.
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Unpacking objects: 100% (3/3), 974 bytes | 974.00 KiB/s, done.
From https://github.com/VerasNp/TesteRepo
   f554917..c137bb7  main      -> origin/main
Updating f554917..c137bb7
Fast-forward
 footer.md | 1 +
 1 file changed, 1 insertion(+)
 create mode 100644 footer.md
```

Observe os commits com `git log`.

Referências

- Git - Uma Breve História do Git. Disponível em: <<https://git-scm.com/book/pt-br/v2/Come%C3%A7ando-Uma-Breve-Hist%C3%B3ria-do-Git>>.
- Sobre repositórios - GitHub Docs. Disponível em: <<https://docs.github.com/pt/repositories/creating-and-managing-repositories/about-repositories>>.